QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	JadualIQ - Web based AI decision copilot for Malaysian part-time and gig workers
Team Repo URL	https://github.com/priasoloitu/UMhackathon.git
Project Board URL	https://jadualiq.atlassian.net?continue=https%3A%2F%2Fjadualiq.atlassian.net%2Fwelcome%2Fsoftware&atlOrigin=eyJpIjoiZTllN2Q5ODE1OWJmNDA0YTkzMzIxZGRiMGVjYjkwMjYiLCJwIjoiaiJ9
Live Deployment URL (optional for preliminary round)	https://u-mhackathon-812r.vercel.app/

Objective:

The primary objective is to ensure that JadualIQ can reliably function as an AI powered decision intelligence system that starts from the user's existing schedule and recommends same day changes. It must successfully optimize for cheaper transport than the original plan, requiring approval before applying changes and validating against the original schedule as a baseline.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- Intake and Clarification parsing, unstructured schedule text parsing via Z.AI GLM
- Context Aware Orchestration, weather triggered surge pricing alerts and traffic based route optimization
- Conflict Resolution, Deterministic validation against database constraints to block impossible schedule moves
- Explanation Panel (Impacts Analytics), tracking quantifiable metrics like hours saved and RM saved

1.2 Out-of-Scope

- Mobile app development (limited to responsive web)
- Production grade gig platform integration (e.g. FoodPanda APIs)
- Automated payroll processing
- Long term analytics
- Production grade public transport integration
- Multi user collaboration.

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
GLM hallucinates travel cost, schedule time, or weather-linked recommendation details	3	5	15 (High)	A deterministic guardrail/validator layer enforces time/number consistency against the database baseline.	Inject conflicting constraints into prompts and verify the system shows notification regarding the conflict
Z.AI GLM API Rate Limiting/Timeout	3	4	12 (High)	Implement 2-attempt retry logic with a 30s-45s timeout, falling back to a static degradation state if unreachable.	Simulate API unreachability and verify the frontend displays a graceful error state without crashing.
External API Failure (Weather/Google Maps)	3	4	12 (High)	Fallback to simulated data or partial processing if APIs fail or return incomplete data.	Use mocked API responses to force a timeout and verify the GLM still returns a limited recommendation.
Uninteded Schedule Overwrites	2	5	10 (Medium)	The AI is the decision maker but changes require explicit user approval before committing to the database	Attempt to bypass approval via direct API calls; verify changes are rejected

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact

© UMHackathon 2026

Email: umhackathon@um.edu.my

4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	**Risk Level**	**Recommended Action**
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - Scope : Testing individual backend components in isolation. This includes validating keyword filtering in the AI guardrail agent, verifying SQLitee database CRUD logic for schedule tasks and restrictions, and testing the core API routing and authentication logic.
  - Execution : Backend tests are written using a Python testing framework utilizing a local test client. Execution happens locally during development and in the CI pipeline
  - Isolation : External API dependencies are strictly mocked. Specifically external calls to weather and traffic mapping services are simulated. Database tests use an isolated in memory SQLite database instead of the production database file.
  - Pass Condition : Achieved expected outcome for happy paths, correctly handled negative cases and handled edge cases without throwing server errors

- **Integration Test**
  - Scope : Verifying the data flow from the API endpoints down to the database and external API agents. The key focus is on the main conversational pipeline (Frontend request – Intake Agent – Guardrail – Weather/Traffic Agent – Main Orchestrator (Z.AI GLM) – Final JSON Response)
- **Test Environment (CI/CD Practice):**
  - Local Testing : Manual UI testing via the browser (verifying the main client application and authentication views) and local API testing using tools like Postman against the development server

- **Regression Testing & Pass/Fail Rules:**
  - Execution Phase : Regression tests run across the core functional endpoints(chat generation, schedule management, restrictions, and impact analytics) each time a new feature branch is merged to the main branch.
  - Pass/Fail Condition: A test case pass is only considered when the actual JSON response perfectly aligns with the expected schema. If the AI orchestrator generates unparseable or hallucinated data that bypasses the guardrail layer the test status is considered a fail.
  - Continuation Rule: The initial deterministic unit tests (database CRUD, auth routing) must meet the passing threshold to allow the pipeline to continue to the more complex orchestrator integration testing.

- **Test Data Strategy:**
  - Manual : Created test accounts representing different worker profiles to manually test the frontend user experience
  - Automated: Pre-populated SQLite database seed files containing baseline schedule data and predefined restriction sets to ensure automated tests have a consistent starting state. Simulated JSON responses are used to mock the outputs of the external weather and traffic agents

© UMHackathon 2026

- **Passing Rate Threshold:**
    - Overall Execution : A minimum of 85% of all executed non critical test cases (such as minor UI rendering issues or non essential analytics endpoints) must pass for a build to be considered acceptable for staging
    - Critical Path Exemption : For core critical operations specifically the AI guardrail keyword filtering, the core database CRUD operations, and the AI orchestrator's ability to return strictly parseable JSON without hallucinating schedule times – the passing rate must be 90%

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	**Requirements**	**Project**	**Pass/Failed**
Automatic Build	**Zero Build Error during backend instantiation and SQLite database connection**	**0 errors**	**Passed**
Unit Tests	**91% Passing Rate for database CRUD operations and user authentication routes**	**91%**	**Passed**
Unit Tests (AI Guardrail)	**95% Pass Rate for keyword filtering and safety boundary agent**	**95%**	**Failed**

4.2 Deployment Thresholds (Pushing to Production)

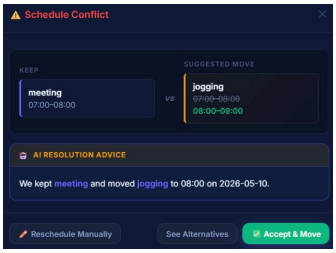
Checks	**Requirements**	**Project**	**Pass/Failed**

© UMHackathon 2026

Email: umhackathon@um.edu.my

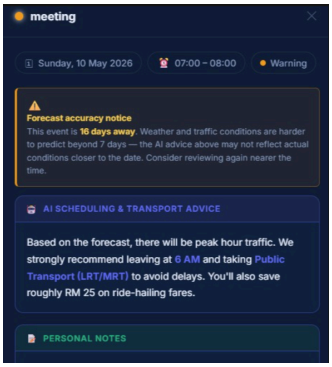
AI Output Pass Rate	**93% of tested GLM recommendations must return valid, strictly parseable JSON formats without bypassing the validation layer**	**93%**	**Passed**
Hallucination Tolerance	**Zero occurrences of the Ai scheduling tasks outside of defined database restrictions**	**0 Occurance**	**Passed**
API Performance	**Standard API routing < 800 ms; Z.AI Orchestrator Generation < 15000ms**	**Met**	**Passed**

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	**Happy Case (Entire Flow)**	Verify the system can parse unstructured text into a valid schedule baseline	1. Send unstructured schedule text via chat interface 2. Wait for Ai parsing 3. Review the parsed UI blocks 4 Click "Approve"	The AI orchestrator successfully structures the schedule into standard time blocks. Upon approval the baseline schedule is successfully saved to the database	Successfully, created a valid schedule based on the prompt that the user sent Status: Passed
TC-02	**Specific Case (Negative):** Conflict Resolution	Test if the deterministic validator blocks impossible schedule moves requested by the AI	1. Add an important meeting to Tuesday 10 AM in the database 2. Ask the AI to move a gig shift to Tuesday 10 AM	The validator intercepts the AI's intent, identifies the database conflict, and recommends a	

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			3. Observe system response	reschedule based on priority of the event	Description: When there are 2 schedules at the same time, the program counts it as conflict and then gives suggestions regarding the conflict. Users can reject and accept Status: Passed
TC-03	**Specific Case (Negative): Decision Intelligence**	test the AI's ability to recommend a cheaper transport option during bad weather to optimize income.	1. Set mocked Traffic Agent to peak hours 2. Request AI to review tomorrow's gig schedule. 3. Check UI for alert.	The Orchestrator evaluates alternative transport, prompts a Forecast alert, and recommends a cheaper alternative.	 Description: The AI gives suggestions about future schedules, in order to save cost. There is also the notice that forecasts of weather beyond 7 days are less accurate. Status: Passed
TC-04	**NFR:** Action Safety	Verify that AI require user confirmation before applying changes	1. Users enter a schedule for the week. 2. Observe user have to approve schedule	After the user input is processed by the Z.AI GLM the system requires user confirmation before proceeding any changes to the schedule.	The system requires confirmation before proceeding. Status: Passed
TC-05	**NFR :** Performance(L	Measure the total end-to-end	1. Submit a simple	The system parses text,	The system parses text, queries mocked external

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
	Latency)	processing time of a standard AI rescheduling request.	schedule request via chat. 2. Measure response time via network panel. 3. Verify response schema integrity.	queries mocked external agents, generates a GLM response, and returns JSON in under 15000ms.	agents, generates a GLM response, and returns JSON in under 12000ms. Status: Passed

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

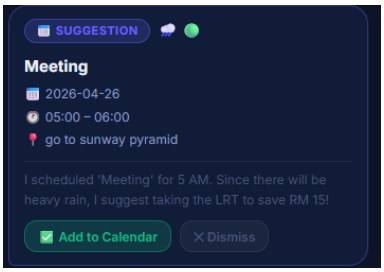
6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	“Create me a schedule”	**System prompts for missing info:** What schedule you wanted to create	“What would you like to schedule? (e.g. meeting,doctor appointment, study session)”	Pass
AI-02	“Hello can you help me with my maths	System rejects safely:	"I can only help you schedule	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

	questions, i need to solve this quadratic equation”	I cannot do unrelated job	your activities. Try something like: - 'schedule a meeting on Wednesday 3pm' - 'I'm free in Thursday morning' - 'block Friday afternoon for study' - 'jadualkan mesyuarat hari rabu pukul 3 petang'"	
AI-03	“Schedule a meeting on Sunday, 26 Apr at 5 AM. I will need to go to Sunway Pyramid, and i will come from monash university malaysia”	System processes integration: Give a scheduling suggestion based on time and location		Pass

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	100 Words/499 Characters/ Approx 1040 Tokens
Input used while testing	1000 Word
Expected Behavior	The system takes the input, prioritizes the most recently recognized time/date entities, and triggers the Clarification Agent to confirm the specific task.
Actual Behavior	The system ignores every character after the 499th character and generated out of context content
Status	Fail

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

In this section, test minimum of 1 with one prompt that is ambiguous, malformed - document how GLM does respond and how your system is going to handle it.

Case 1 (When user ask the chatbot things that are out of scope from scheduling issue)
Prompt: when the input is just too out of context aside from the scheduling stuff, for example, “Hey can u help me do my math question”
Answer: the chatbot will return, “I can only help you schedule your activities”

Case 2 (Testing using unavailable time and date)
Prompt: Schedule something on Tuesday, 32 April at 25 AM
Answer: Throw error and ask user to reprompt again.

6.4. Hallucinating Handling

To detect and mitigate hallucinated responses, the system employs a multi-agent architecture in which multiple agents independently validate and cross-filter generated outputs. This layered verification process ensures that only factually grounded and reliable data is passed through, reducing the likelihood of fabricated or inaccurate information reaching the end user.

References:

- Thorhauge, M., Swait, J., & Cherchi, E. (2020). The habit-driven life: Accounting for inertia in departure time choices for commuting trips. Transportation Research Part A: Policy and Practice, 133, 272–289. https://doi.org/10.1016/j.tra.2020.01.023
- Wong, A. (2022, May 27). Why is eHailing so expensive? Here’s what Grab and AirAsia Ride have to say about surge pricing. SoyaCincau. https://soyacincau.com/2022/05/27/why-is-ehailing-malaysia-so-expensive-these-days-grab-and-airasia-ride-responds/

© UMHackathon 2026

Email: umhackathon@um.edu.my